ENEE 633 - Statistical Pattern Recognition Project 01

Kiyan Amirian

April 15, 2025

In this report, we present the results of analyzing the dataset DATA. The dataset is first divided into training and test sets before being passed to the classifiers. Dimensionality reduction techniques — PCA and MDA — are applied using the functions compute_pca and compute_mda, respectively. These are fitted on the training set, and the test set is projected onto the components derived from the training data.

For Task 1, we use the first two images of each subject as the training data, and the third image as the test data, using the function separate_train_test_manual. In contrast, Task 2 discards the third image of each person and treats the remaining 400 images (from 200 subjects × 2 images) as the total dataset. A random subset is then selected for training and testing.

Both PCA and MDA are implemented as functions. For Task 1, the number of PCA and MDA components can be specified and varied across experiments. This allows us to analyze how the number of components affects classification accuracy for each classifier.

Bayes classifier

The result for bayes classifier can be seen in the image below for task 1:

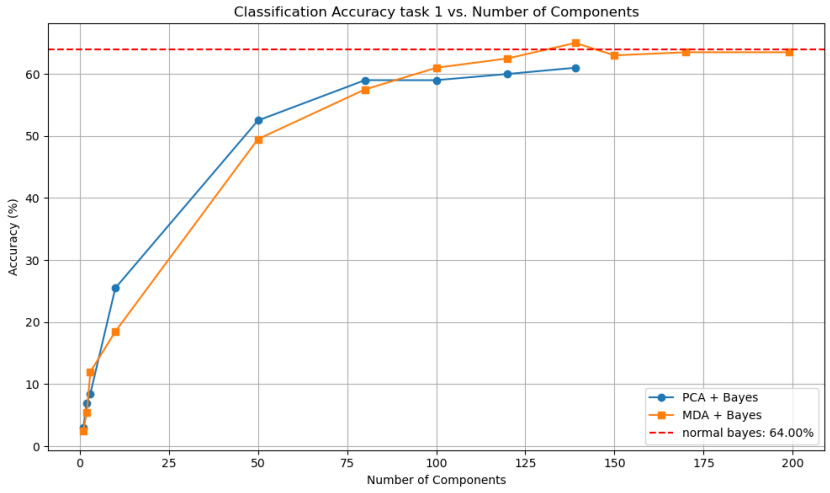

Figure 1: Bayes classifier for task 1.

MDA + Bayes surpassed the baseline Bayes at 139 components and outperformed PCA + Bayes after 100 components, showing stronger discriminative power with more components.

For task 2, as shown in the figure, the MDA classifier outperforms both PCA and standard Bayes by about 10% in accuracy. Notably, more PCA components do not always lead to better performance—highlighting the effects of the curse of dimensionality and the potential for added noise in higher dimensions, as discussed in class.

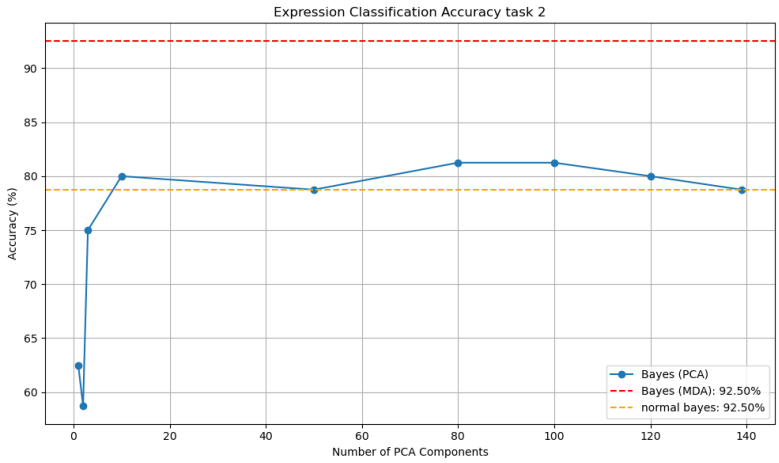

Figure 2: Bayes classifier for task 2

k-NN

For k-NN, classification was based on majority voting among the k nearest neighbors using Euclidean distance. In case of a tie, the average distance of the tied neighbors was calculated, and the label of the group with the closest average distance was selected. As shown in the left figure, k=3 yielded the highest accuracy for Task 1, which involves 200 classes. This makes sense, as a small k helps avoid including too many misleading neighbors from other classes, which is more likely with a high number of classes.

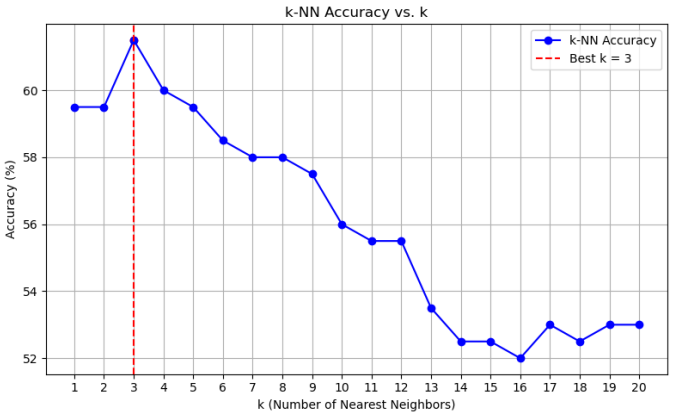

Figure 3: k-NN task 1.

With PCA and MDA applied (right figure), performance generally improved across all k values. Notably, 1-NN becomes nearly as effective as 3-NN, especially with MDA, which is expected as

MDA optimizes class separability. PCA also enhances performance, though its effect is slightly less consistent. These results highlight how dimensionality reduction techniques can improve robustness, particularly in high-dimensional spaces.

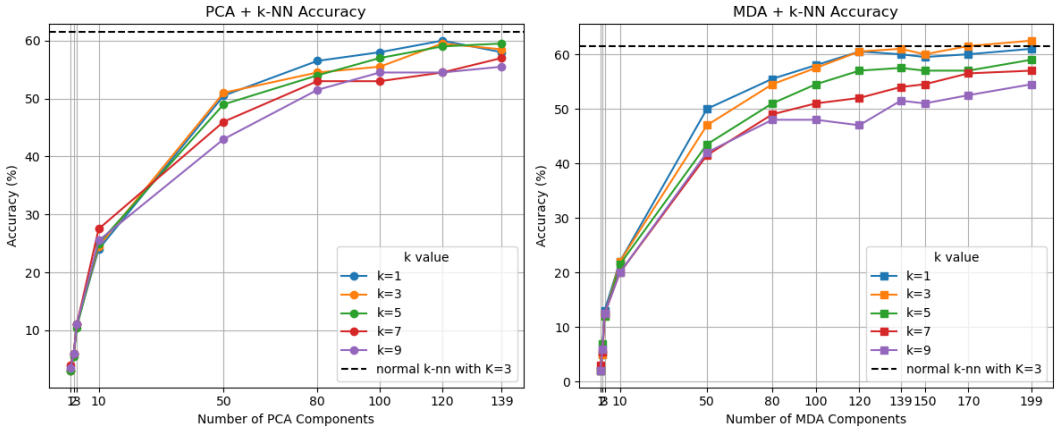

Figure 4: k-NN task 1 with PCA and MDA.

For task 2, the accuracy steadily increases with larger values of k, indicating that the model benefits from aggregating more neighbors when dealing with only two classes. This behavior contrasts with task 1, where a smaller k performed better due to the larger number of classes and potential label noise from distant neighbors.

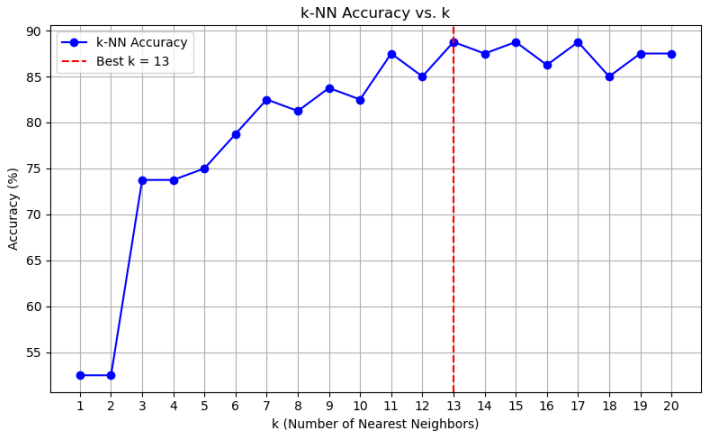

Figure 5: k-NN task 2.

As shown in the PCA and MDA plots, MDA continues to significantly boost k-NN performance. Even with just 1-NN, MDA surpasses the best standard k-NN baseline, highlighting its ability to extract highly discriminative features. However, for PCA, the improvement is not as consistent—especially at lower k, where performance plateaus or declines beyond certain component counts. This reinforces the earlier observation that more PCA components don't always yield better results due to the curse of dimensionality or increased noise.

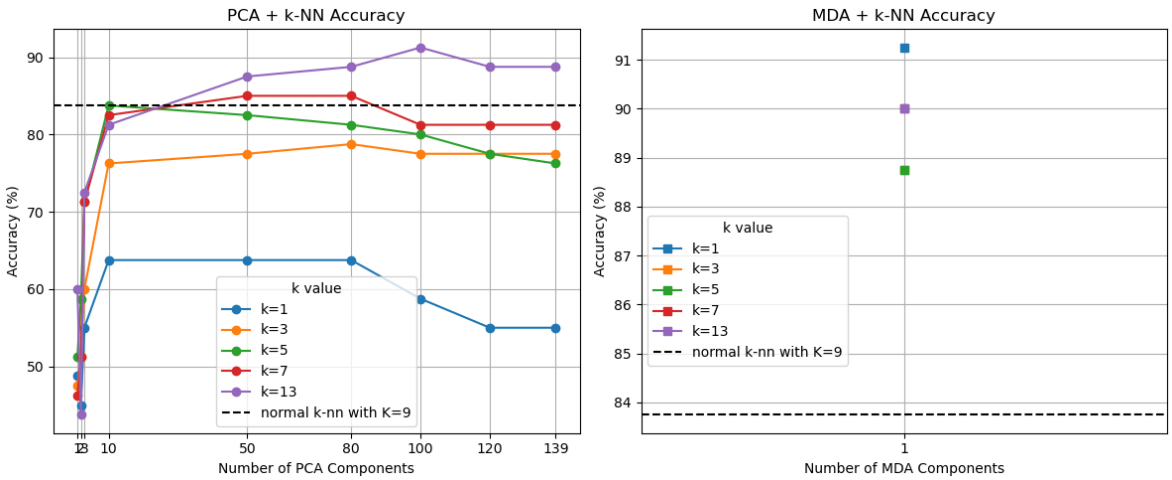

Figure 6: k-NN task 2 with PCA and MDA.

Kernel SVM

For the kernel SVM, I initially implemented a solver using CVXOPT, which is well-suited for solving quadratic programming problems like the dual form of SVM. In this setup, the dual objective:

$$\max_{\lambda} L_D = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_i y_i y_j K(x_i, x_j) \ \ subject\ to \ \sum_i \lambda_i y_i = 0\,, \lambda_i \geq 0$$

Is reformulated into the standard quadratic form required by cvxopt:

$$\min_x \frac{1}{2} x^T P x + q^T x \ \ subject\ to\ Gx \leq h, \qquad Ax = b$$

Where $x \equiv \lambda$, and $p_{ij} = y_i y_j K(x_i, x_j)$ encodes the kernel matrix weighed by class labels. The constraints enforce margin and non-negativity conditions.

However, when using the polynomial kernel, I observed that the optimizer returned zero solutions for degrees greater than 3. This issue arises due to numerical instability in the kernel matrix, which becomes poorly conditioned as the polynomial degree increases—leading to convergence failures in CVXOPT as seen in figure 7.

To overcome this, I switched to a gradient descent-based SVM solver, which iteratively updates the dual variables using gradients and clipping. While slower, this method is more robust to numerical issues and works reliably even with high-degree polynomial kernels, making it a better fit for this scenario.

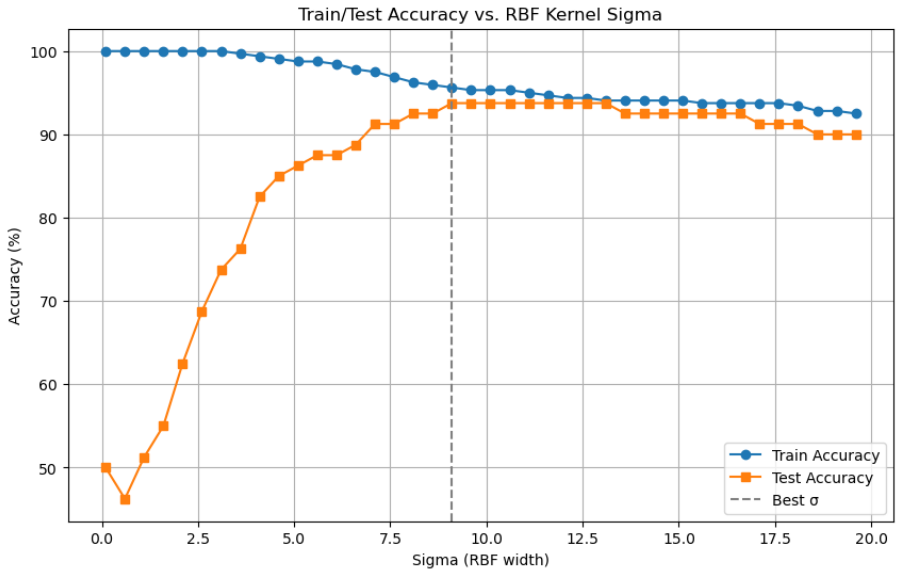

Figure 7: RBF kernel SVM with cvxopt.

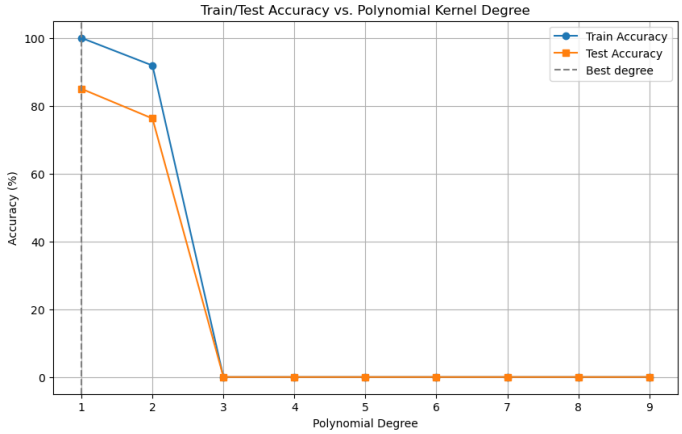

Figure 8: polynomial kernel SVM with cvxopt.

The figures below show the result of training accuracy vs test accuracy of the SVM kernel using gradient descent:

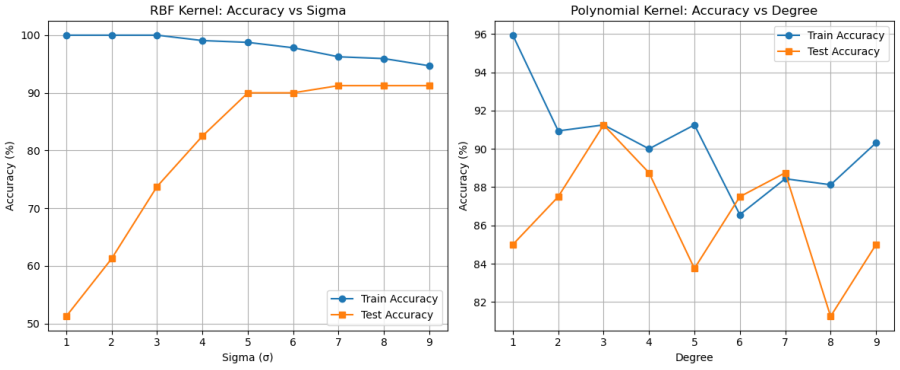

Figure 9: polynomial and RBF kernel SVM with gradient descent.

In the left plot (RBF Kernel), we observe that with very small sigma values (e.g., σ = 1), the model overfits — achieving nearly perfect accuracy on the training set but poor performance on the test set. As sigma increases, the kernel becomes smoother and less sensitive to individual data points, leading to better generalization and improved test accuracy. Beyond σ ≈ 5, the test accuracy stabilizes, indicating a good balance between bias and variance.

In the right plot (Polynomial Kernel), increasing the degree introduces more model complexity. As expected, test accuracy fluctuates and generally declines beyond degree 3, which aligns with the tendency of high-degree models to overfit. Interestingly, the training accuracy also drops at higher degrees, which is somewhat unexpected — in theory, a higher-degree polynomial should better fit the training data. This drop may be due to numerical instability or poor optimization convergence with high-degree polynomial features. The gradient descent parameters used were D=1, learning rate=0.001.

To select the best values for the RBF kernel's σ and the polynomial kernel's degree, cross-validation was performed using the k-fold method. In this approach, the training data is split into k subsets (I used k=5); each subset takes a turn as the validation set while the others form the training set. The average accuracy across all folds is then used to evaluate each hyperparameter setting.

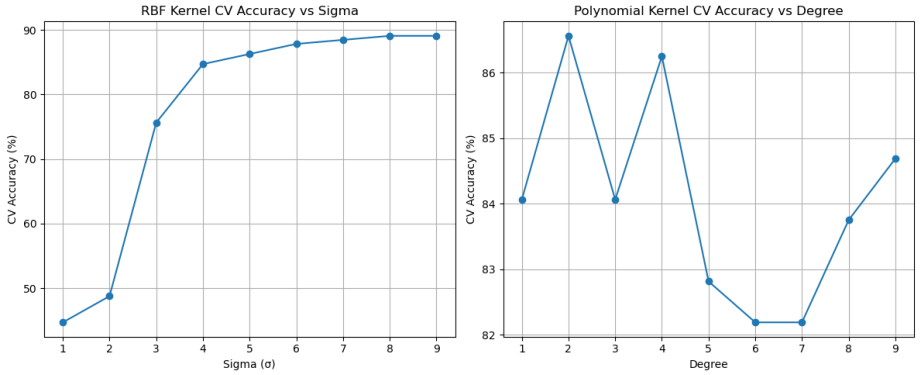

Figure 10: cross validation for polynomial and RBF kernel SVM with gradient descent.

As shown in the figure, cross-validation results for the RBF kernel follow a similar trend to the test accuracy in Figure 8—indicating that higher σ values (around 8–9) lead to better generalization. For the polynomial kernel, although degree 3 does not yield the highest cross-validation accuracy, it shows stronger performance on the actual test set in Figure 8. Therefore, degree 3 is likely a better choice despite slightly lower validation performance.

Using the optimal hyperparameters (σ = 9 for RBF and degree = 3 for polynomial), both models were evaluated across varying PCA components and compared to MDA as can be seen in figure below:

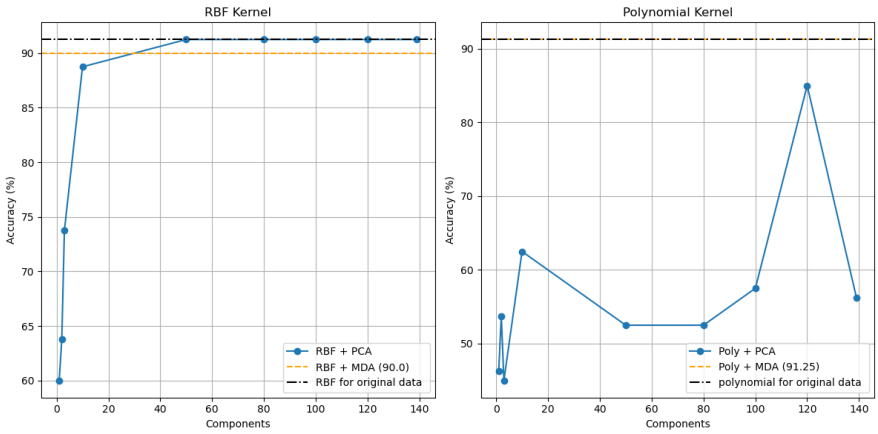

Figure 11: influence of PCA and MDA polynomial and RBF kernel SVM with gradient descent.

RBF Kernel: Accuracy improves rapidly with PCA and plateaus around 60 components, reaching ~91% matching the full feature set. Even 10 components outperform MDA, suggesting RBF benefits from compact, denoised features and is robust to dimensionality reduction. Polynomial Kernel: PCA performance is less stable, with fluctuations and poor results at lower dimensions. Only around 120 components do it approach good accuracy. In contrast, MDA is consistently strong (91.25%), indicating better alignment with the kernel's sensitivity to feature distributions.

SVM + Boosting

To evaluate the impact of boosting, a linear kernel SVM was used as the weak learner within the AdaBoost framework. At each round, the training samples are weighted according to their classification difficulty—misclassified points receive higher weights, increasing their likelihood of being sampled in subsequent rounds. A new SVM is trained on this weighted subset, and the corresponding classifier weight (α) is computed and stored. The final decision is made by aggregating all weak learners weighted by their α values.

Figure 11 shows the accuracy progression over 20 boosting rounds with c=1, learning rate=0.001. The training accuracy quickly reaches 100%, while test accuracy improves steadily, peaking around round 6, and then slightly oscillates—indicating potential mild overfitting. Nevertheless, the boosting approach maintains high generalization performance (~91–92%) across most rounds. But the fluctuation is a bit strange, and I do not know the specific reason.

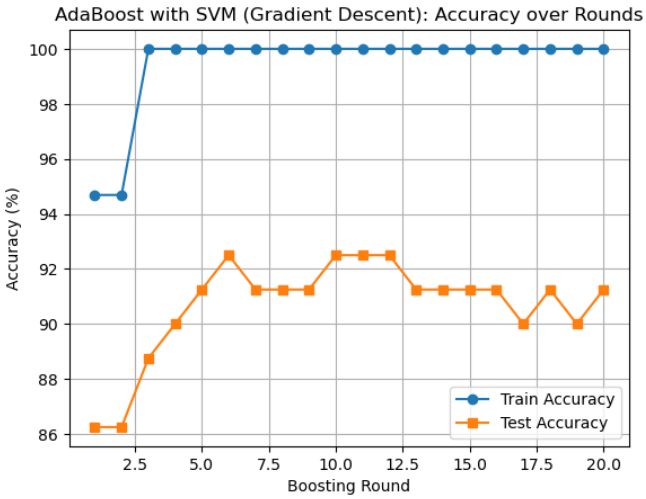

Figure 12: ADA boost with linear SVM with gradient descent.

Now the influence of PCA and MDA is evaluated on AdaBoost with a linear SVM. PCA improves performance up to ~94% around 50 components, after which accuracy gradually declines—likely due to added noise from less informative dimensions. MDA, on the other hand, achieves a stable 90% accuracy using a compact label-aware projection. While PCA can outperform MDA when well-tuned, MDA provides consistently strong results without the need for component tuning.

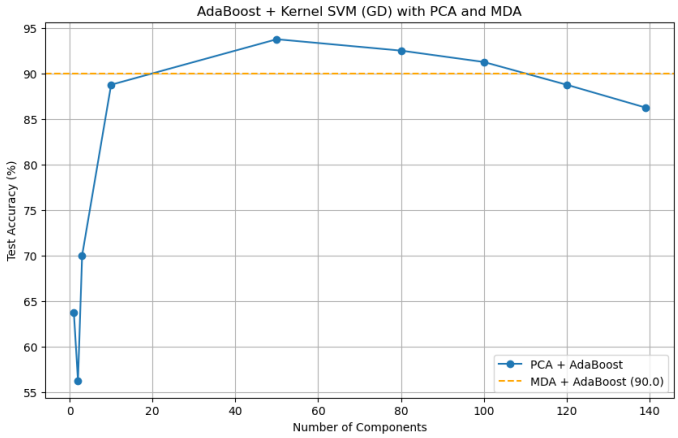

Figure 13: influence of PCA and MDA on ADA boost with linear SVM with gradient descent.

Conclusion:

Each classifier benefits differently from dimensionality reduction: MDA consistently improves performance in tasks involving many classes or when paired with linear models, while PCA can be more effective with non-linear kernels such as RBF when properly tuned. Notably, dimensionality reduction via PCA or MDA significantly reduced computational time, making training and evaluation much faster compared to using the full feature space.

Among all methods tested, for Task 1 (200-class identity recognition), the AdaBoosted linear SVM with PCA achieved the highest accuracy (~94%), demonstrating the strength of combining boosting with compact representations. For Task 2 (expression classification with 2 classes), the RBF kernel SVM reached the best performance (~91.25%) using either the full feature set or PCA.

Boosting further enhanced the linear SVM's performance across both tasks, showing its ability to refine even simple learners through adaptive weighting and ensemble learning.